function in listing 5.9; the main difference is that this filter handles a variable location of the `id` parameter.

Listing 5.11 Using a filter factory to create an endpoint filter

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();
var _fruit = new ConcurrentDictionary<string, Fruit>();

app.MapGet("/fruit/{id}", (string id) =>
    _fruit.TryGetValue(id, out var fruit)
        ? TypedResults.Ok(fruit)
        : Results.Problem(statusCode: 404))
    .AddEndpointFilterFactory(ValidationHelper.ValidateIdFactory);      ❶

app.MapPost("/fruit/{id}", (Fruit fruit, string id) =>
    _fruit.TryAdd(id, fruit)
        ? TypedResults.Created($"/fruit/{id}", fruit)
        : Results.ValidationProblem(new Dictionary<string, string[]>
          {
              { "id", new[] { "A fruit with this id already exists" } }
        }))
    .AddEndpointFilterFactory(ValidationHelper.ValidateIdFactory);      ❶

app.Run();

class ValidationHelper
{
    internal static EndpointFilterDelegate ValidateIdFactory(
        EndpointFilterFactoryContext context,                           ❷
        EndpointFilterDelegate next)
    {
        ParameterInfo[] parameters =                                    ❸
            context.MethodInfo.GetParameters();                         ❸
        int? idPosition = null;
        for (int i = 0; i < parameters.Length; i++)                     ❹
        {                                                               ❹
            if (parameters[i].Name == "id" &&                           ❹
                parameters[i].ParameterType == typeof(string))          ❹
            {                                                           ❹
                idPosition = i;                                         ❹
                break;                                                  ❹
            }                                                           ❹
        }                                                               ❹
```